

Reto Práctico #4 — Modernización del Core Bancario Modular (FinBank)

Documento de Arquitectura: Diagramas C4, ADRs y Análisis de Trade-offs

Diplomado — Módulo 4: Arquitectura de Microservicios y Sistemas Distribuidos

Junio 2026 - Realizado por Jhonatan Escobar Uribe

Tabla de contenido

Presentación del documento	3
Mapa de entregables por paso	3
Arquitectura	3
Punto de partida — Monolito Modular (antes)	3
Dependencias entre módulos	4
Capas internas de cada módulo	5
Aislamiento de schemas en PostgreSQL	6
Arquitectura objetivo — Modelo C4 (después)	7
Nivel 1 — Diagrama de Contexto	8
Nivel 2 — Diagrama de Contenedores	9
Nivel 3 — Diagrama de Componentes (ms-transfers)	10
Arquitectura de Eventos (Paso 3)	11
Catálogo	11
Flujo completo — Happy path	12
Flujos de fallo	13
Patrones de resiliencia implementados (6)	14
Decisiones de Arquitectura (ADR)	14
ADR-001 — Elección del primer módulo a extraer: notifications	14
ADR-002 — Elección del segundo módulo a extraer: transfers	15
ADR-003 — API Gateway: Spring Cloud Gateway	16
ADR-004 — Message Broker: Apache Kafka	17
ADR-005 — Base de datos de ms-notifications: PostgreSQL 16 dedicado	18
ADR-006 — Base de datos de ms-transfers: PostgreSQL 16 dedicado	18
ADR-007 — Patrón de consistencia distribuida: Saga orquestada con movimiento de dinero atómico	19
ADR-008 — Arquitectura interna de los microservicios: Hexagonal pragmática (sin CQRS)	20
ADR-009 — Stack de observabilidad: OpenTelemetry + Jaeger + Prometheus/Grafana + Loki	21
ADR-010 — Contrato de eventos: JSON con envelope CloudEvents y versionamiento explícito	22
ADR-011 — Estrategia de migración de datos sin downtime: Expand → Migrate → Contract	23
ADR-012 — Microfrontends: composición con Module Federation (Webpack 5)	24

Análisis de Trade-offs (Paso 5)	26
1. ¿La consistencia eventual es aceptable para todas las operaciones bancarias de FinBank?	26
2. ¿En qué operaciones se sacrificó disponibilidad para mantener consistencia?	26
3. ¿Qué tan difícil sería revertir las decisiones? ¿Cuáles son reversibles?	26
4. ¿El incremento en complejidad operativa está justificado para el tamaño actual de FinBank?	27
5. ¿Hubo módulos que se decidió NO extraer? ¿Por qué?	27
Anexo — Evidencias funcionales (capturas)	28
Paso 3 — Resiliencia (demos)	28
Paso 4 — Observabilidad y trazabilidad	28
★ Microfrontends	29

Presentación del documento

FinBank es un banco digital cuyo core operaba como un **Monolito Modular** (Spring Boot 3) con fronteras lógicas claras entre dominios. Este documento formaliza la modernización progresiva hacia microservicios aplicando el patrón **Strangler Fig**, junto con la comunicación asíncrona, la resiliencia y la observabilidad distribuida exigidas por el reto. Se estructura siguiendo los pasos de la guía.

Mapa de entregables por paso

Paso de la guía	Contenido en este documento
Paso 1 — Primera extracción (Strangler Fig)	§2 Arquitectura · ADR-001, ADR-003, ADR-005, ADR-011
Paso 2 — Segunda extracción + comunicación asíncrona	§2 Arquitectura (C4) · ADR-002, ADR-004, ADR-006, ADR-007, ADR-008
Paso 3 — Resiliencia y contratos de eventos	§3 Arquitectura de eventos · ADR-010
Paso 4 — Observabilidad y trazabilidad	ADR-009 (evidencias: anexo de capturas)
Paso 5 — ADR + análisis de trade-offs	§4 Decisiones de Arquitectura · §5 Trade-offs
★ Microfrontends	ADR-012

Arquitectura

Punto de partida — Monolito Modular (antes)

El sistema entregado es un monolito modular con cinco dominios (auth, accounts, transfers, notifications, audit), cada uno con su propio schema y su interfaz interna formal. El módulo transfers orquesta las transferencias invocando **in-process** a accounts, notifications y audit.

Dependencias entre módulos

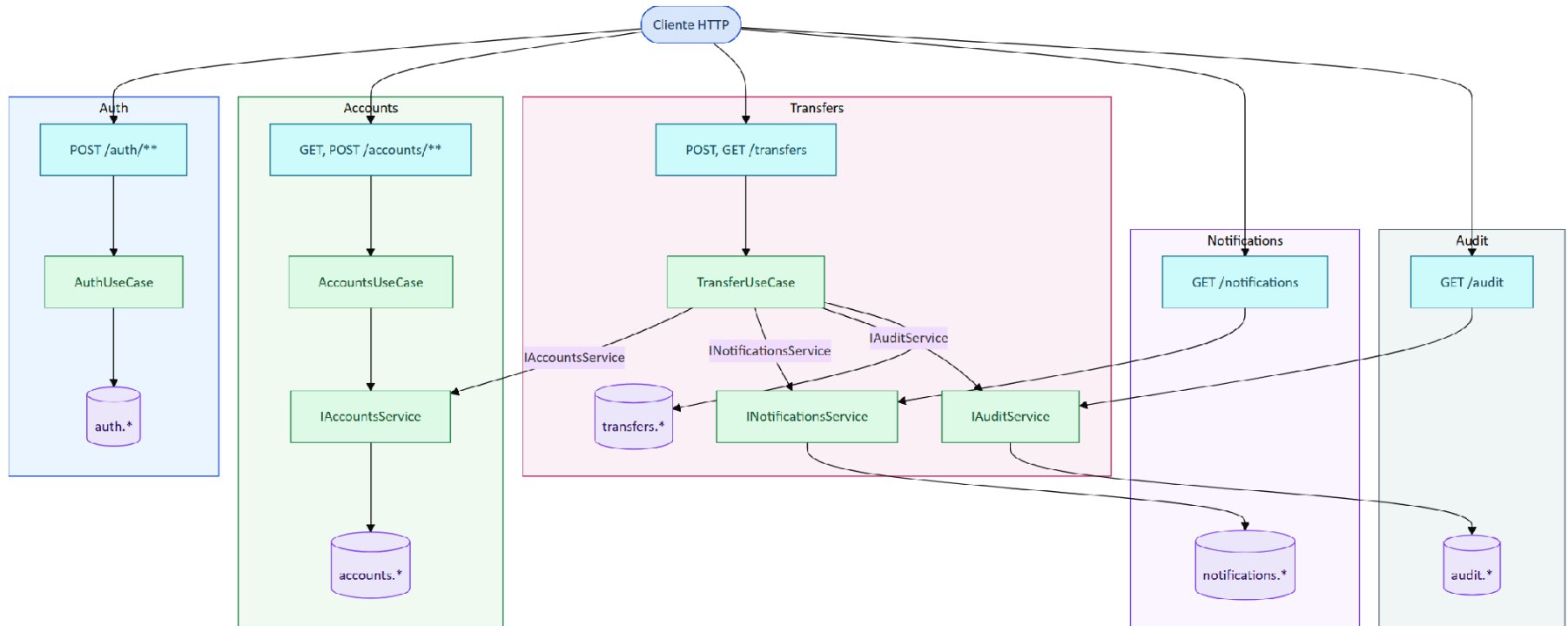


Figura 1 — Dependencias entre módulos del monolito

Capas internas de cada módulo

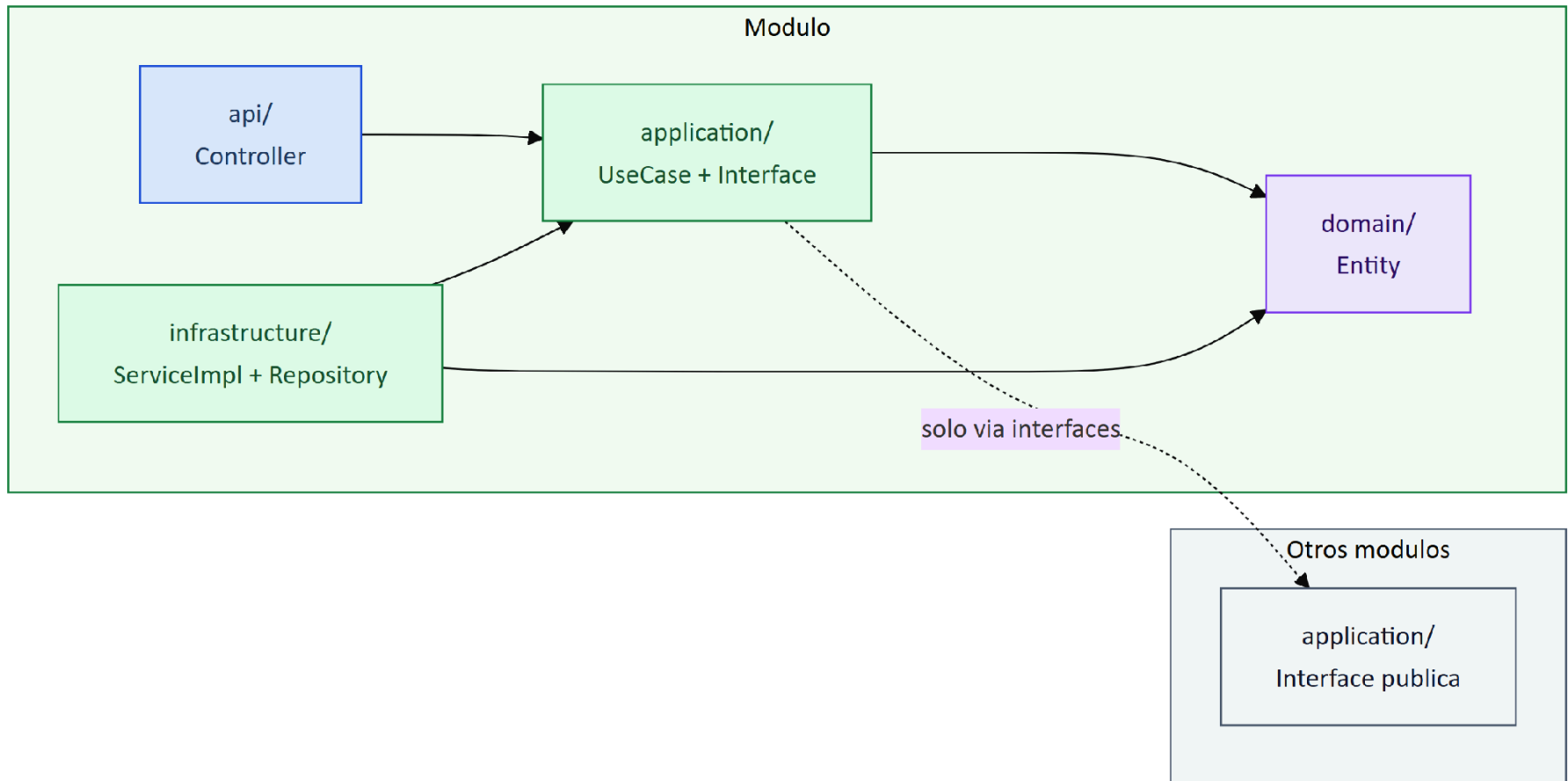


Figura 2 — Capas internas por módulo

Aislamiento de schemas en PostgreSQL

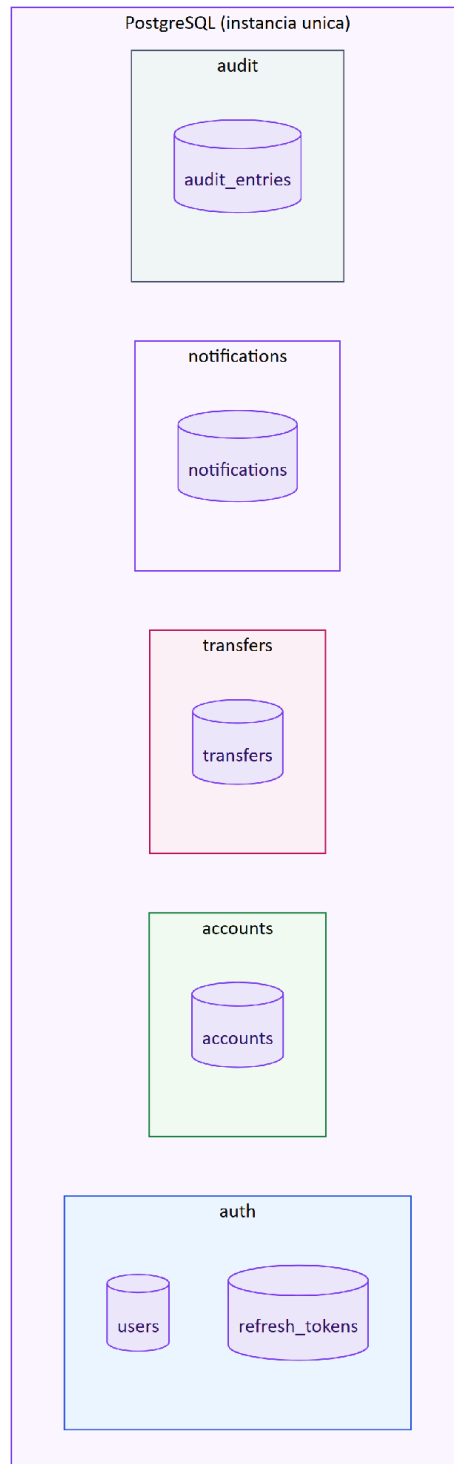


Figura 3 — Schemas aislados en una única instancia

Arquitectura objetivo — Modelo C4 (después)

Tras la migración, el sistema queda compuesto por: portal de microfrontends, API Gateway (Spring Cloud Gateway), dos microservicios extraídos (ms-notifications, ms-transfers), el monolito remanente (auth, accounts, audit) y Kafka como broker. Se documenta con los tres primeros niveles del modelo **C4**.

Nivel 1 — Diagrama de Contexto

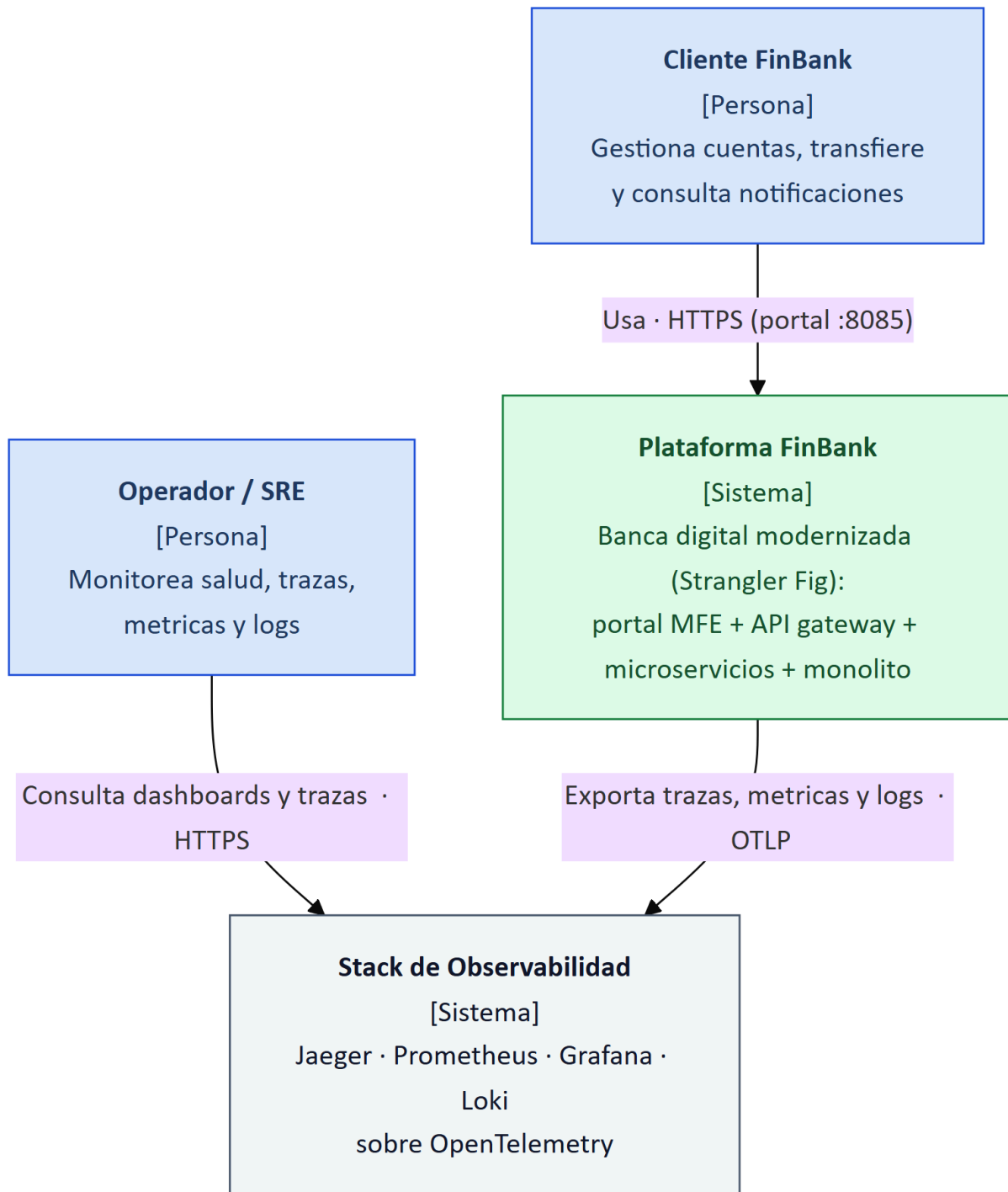


Figura 4 — C4 Nivel 1: Contexto del sistema

Nivel 2 — Diagrama de Contenedores

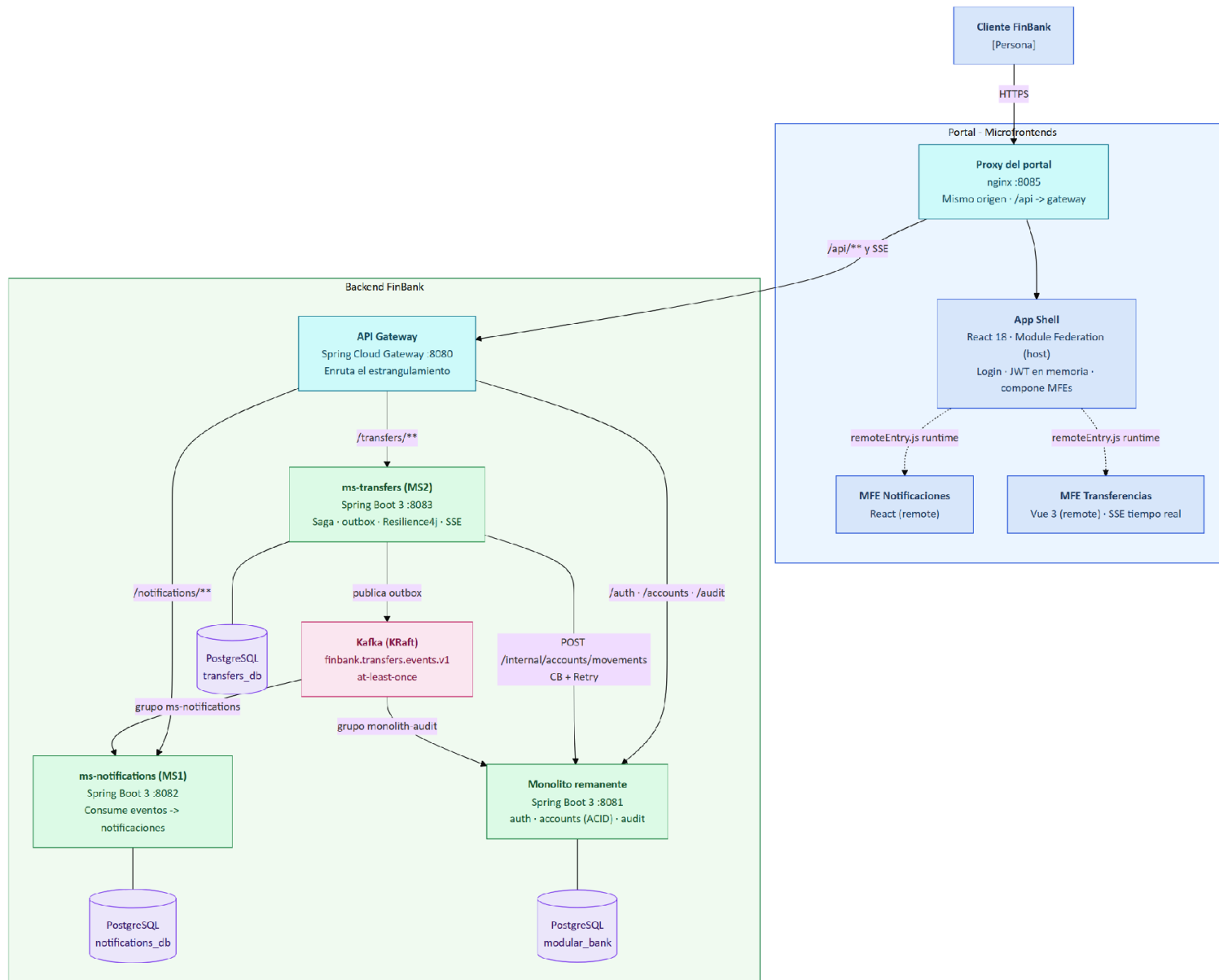


Figura 5 — C4 Nivel 2: Contenedores

Nivel 3 — Diagrama de Componentes (ms-transfers)

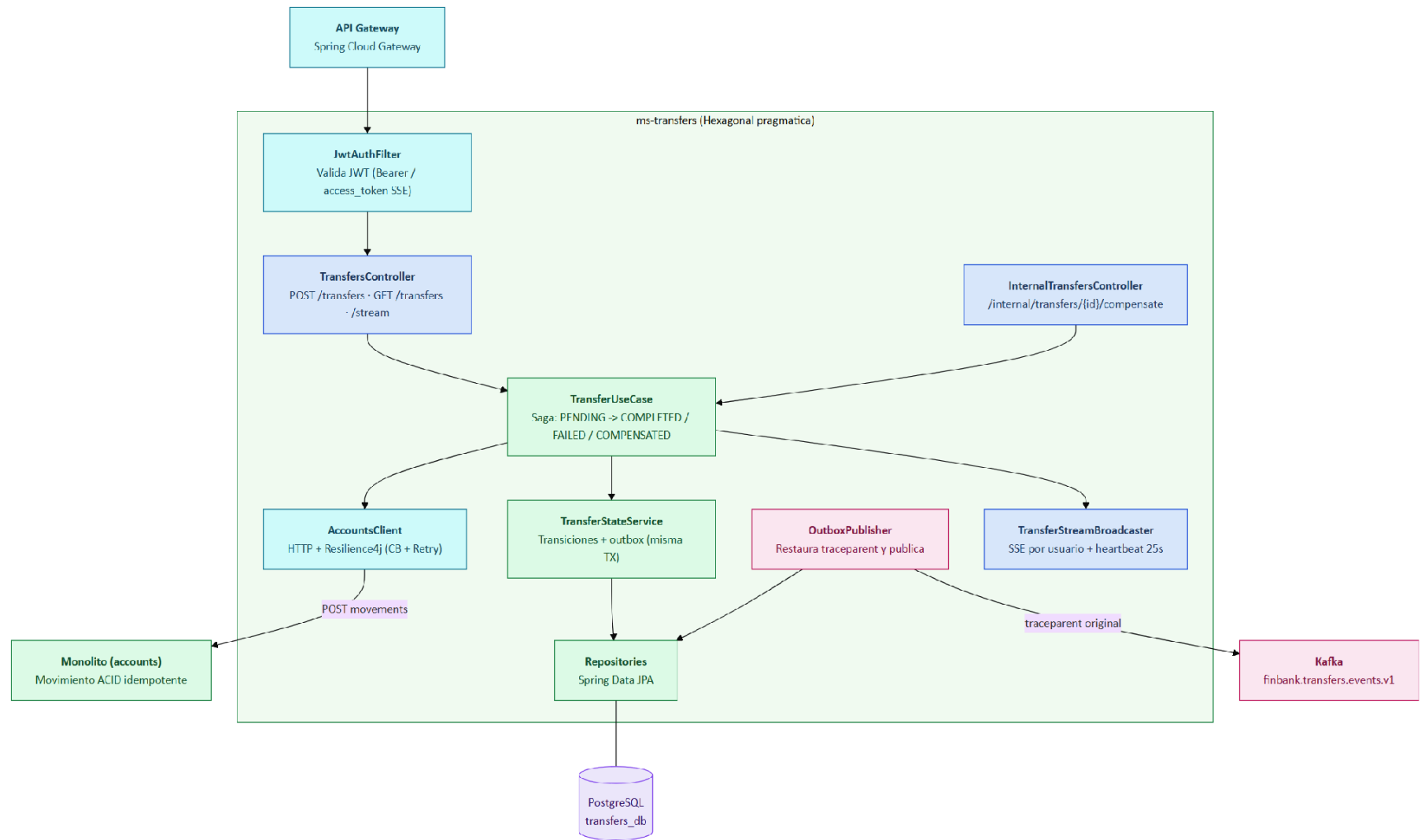


Figura 6 — C4 Nivel 3: Componentes de ms-transfers

Arquitectura de Eventos (Paso 3)

Catálogo

Evento (type)	Productor	Consumidores (group)	Reacción
finbank.transfers.TransferCompleted.v1	ms-transfers (outbox)	ms-notifications (ms-notifications), monolito audit (monolith-audit)	Notificación TRANSFER_SENT / registro TRANSFER_EXECUTED
finbank.transfers.TransferFailed.v1	ms-transfers (outbox)	ídem	Notificación TRANSFER_FAILED / registro TRANSFER_FAILED
finbank.transfers.TransferCompensated.v1	ms-transfers (outbox)	ídem	Notificación TRANSFER_COMPENSATED / registro TRANSFER_COMPENSATED

Tópicos: finbank.transfers.events.v1 (key = transferId → orden por agregado) y finbank.transfers.events.v1.dlt (dead letter). Contrato y versionamiento: ver ADR-010.

Flujo completo — Happy path

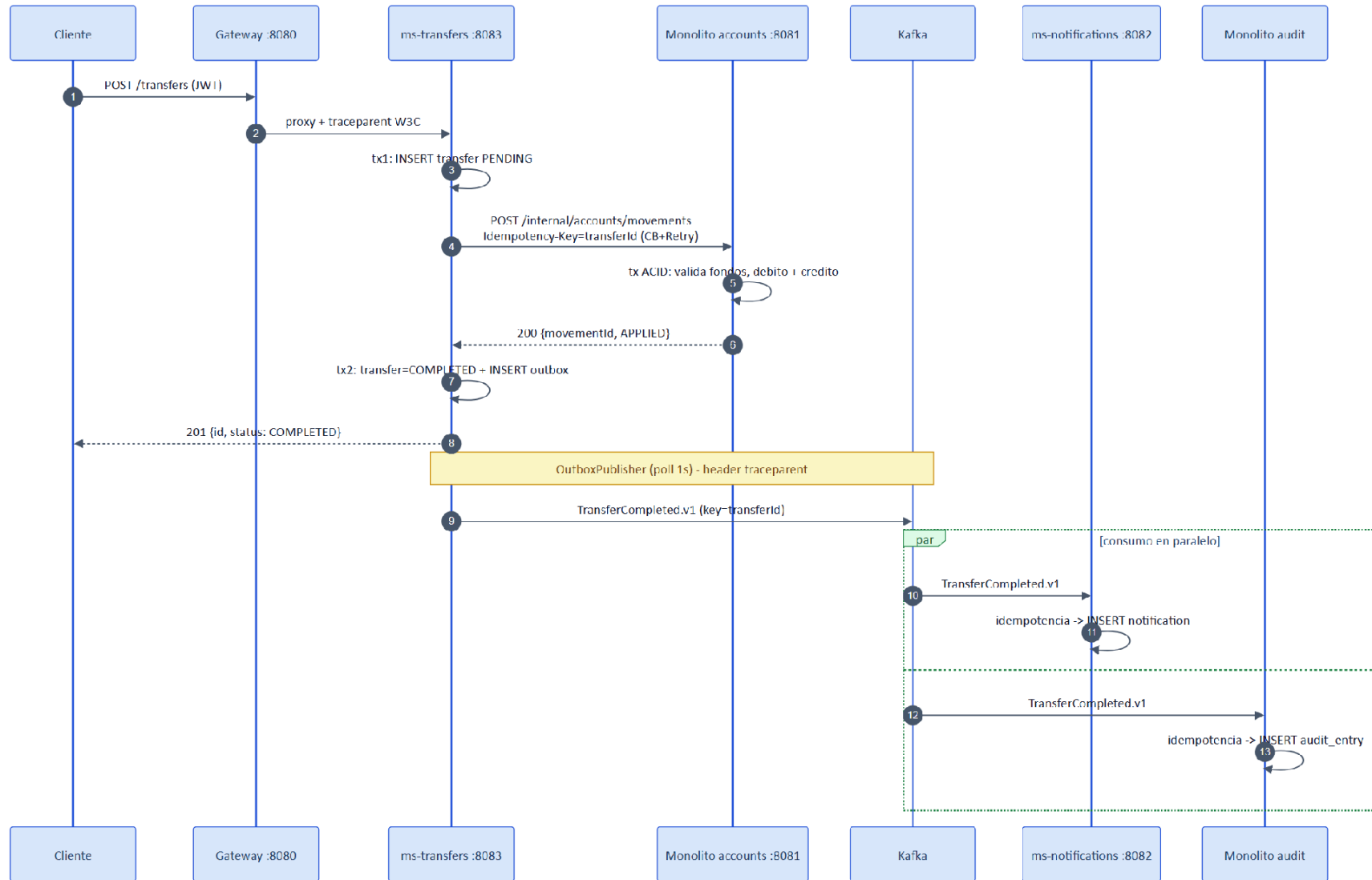


Figura 7 — Secuencia: Happy path

Flujos de fallo

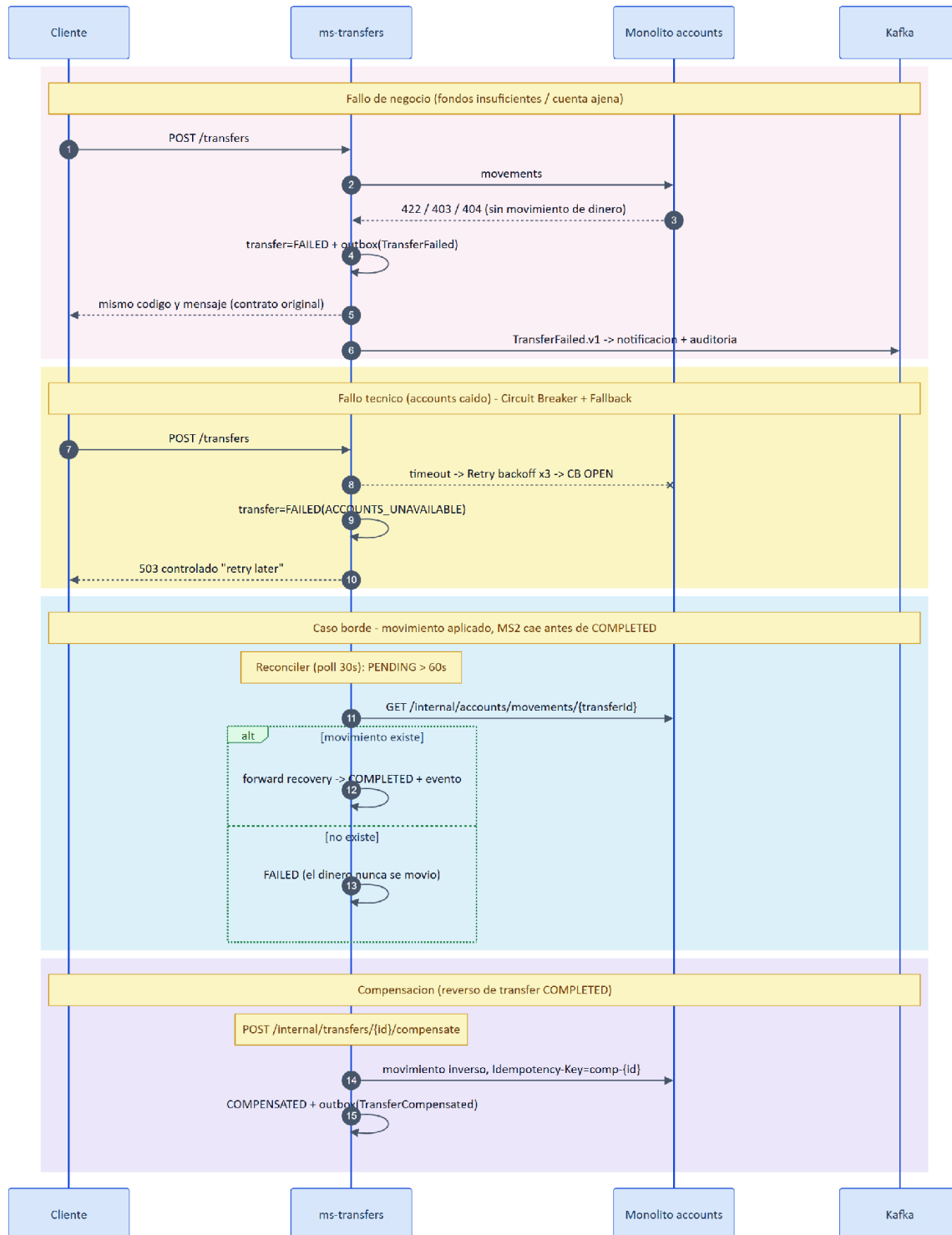


Figura 8 — Secuencia: Flujos de fallo

Patrones de resiliencia implementados (6)

#	Patrón	Dónde está en el código
1	Outbox	ms-transfers/.../domain/OutboxEvent.java, TransferStateService (misma tx), OutboxPublisher
2	Idempotenc y Key	monolith/.../MovementsServiceImpl (processed_movements); consumidores: processed_events en MS1 y monolito
3	Circuit Breaker	ms-transfers/.../AccountsHttpAdapter + config resilience4j.circuitbreaker.instances.accounts
4	Retry + Backoff exponencial	mismo adaptador (resilience4j.retry) y DefaultErrorHandler con ExponentialBackOff en consumidores Kafka
5	DLQ	KafkaConsumerConfig (MS1 y monolito): DeadLetterPublishingRecoverer → ...v1.dlt
6	Fallback / Degraded mode	TransferUseCase: 503 controlado {"message": "Transfers temporarily unavailable..."} con CB abierto

Decisiones de Arquitectura (ADR)

Registro formal de las decisiones técnicas (mínimo 11 exigidas; se documentan 12). Cada ADR sigue el formato Contexto / Opciones evaluadas / Decisión / Consecuencias.

ADR-001 — Elección del primer módulo a extraer: notifications

Estado: Aprobado

Contexto El reto exige extraer un primer módulo del monolito modular con Strangler Fig, justificando la elección por volumen de tráfico, autonomía del dominio, dependencias entrantes/salientes y riesgo de migración de datos. La prioridad declarada del negocio es **estabilidad del servicio y trazabilidad regulatoria durante la transición**.

Opciones evaluadas

Opción	Ventajas	Desventajas
notifications	Módulo hoja: 0 dependencias salientes, 1 entrante (transfers). Tolerancia eventual por naturaleza. Tabla append-mostly sin FKs externas → migración de datos de bajo riesgo. Alto volumen de escrituras (1 por evento de negocio) → beneficio real de escalar independiente.	Bajo “prestigio” arquitectónico: no es el dominio core.
accounts	Es el corazón del negocio; extraerlo primero “resuelve lo difícil”.	Máximo riesgo posible: dinero, consistencia fuerte, todas las transferencias dependen de él. Contradice la prioridad de estabilidad.
transfers	Orquestador con lógica interesante.	Depende de otros 3 módulos: extraerlo primero obliga a resolver 3 integraciones remotas de golpe.

Opción	Ventajas	Desventajas
audit	Sin dependencias salientes.	Trazabilidad regulatoria: el regulador penaliza pérdidas; no conviene tocarlo en la primera incisión.
auth	—	Credenciales y emisión de JWT: riesgo de seguridad alto y es transversal a todo.

Decisión Se extrae **notifications** como MS1. La primera incisión del Strangler Fig debe minimizar el blast radius: es el módulo con mayor autonomía (hoja del grafo de dependencias), cuyo dominio tolera consistencia eventual, y cuya migración de datos es la más simple. Permite validar toda la cadena (gateway, DB exclusiva, contenerización, observabilidad) sobre un dominio de bajo riesgo antes de tocar dominios con dinero.

Consecuencias - (+) Riesgo mínimo en la primera extracción; el contrato GET /notifications se preserva sin cambios para los clientes. - (+) Prepara el terreno del Paso 2: transfers (su único consumidor) será el segundo módulo, cumpliendo la relación de dependencia exigida. - (–) El módulo transfers del monolito debe adaptarse para invocar notifications remotamente (transitorio: en el Paso 2 esa interacción pasa a ser un evento Kafka). - (–) Deuda técnica: el enum NotificationType queda duplicado entre servicios hasta definir una librería de contratos.

ADR-002 — Elección del segundo módulo a extraer: transfers

Estado: Aprobado

Contexto El Paso 2 exige extraer un segundo módulo **que tenga relación de dependencia con MS1 (notifications)** y que se comunique con él de forma asíncrona vía broker, evaluando el impacto en la consistencia de datos.

Opciones evaluadas

Opción	Ventajas	Desventajas
transfers	Es el único módulo que dependía de notifications (NotificationsService.send() in-process) → cumple el requisito literalmente. Es el orquestador del proceso más crítico: extraerlo hace explícita la coordinación distribuida (saga, outbox, compensación), exactamente la competencia evaluada. Al salir, el monolito remanente no depende de ningún microservicio (dirección de dependencia limpia).	Convierte la transacción ACID transferencia en una coordinación distribuida que hay que diseñar con cuidado (ADR-007).
accounts	También se relaciona con transfers.	No depende de notifications (no cumple el requisito) y es el módulo de mayor riesgo; además dejaría al monolito llamando hacia afuera para cada operación de saldo.
audit	Consume eventos naturalmente.	No depende de notifications; el reto pide explícitamente que los módulos restantes consuman eventos <i>sin ser extraídos</i> — audit es el ejemplo perfecto de eso.

Decisión Se extrae **transfers** como MS2. Su dependencia con MS1 (la notificación post-transferencia) se resuelve con el evento TransferCompleted vía Kafka, reemplazando la llamada in-process. Su dependencia con accounts (dinero) permanece **síncrona** contra el monolito con consistencia fuerte (ADR-007). El módulo audit permanece en el monolito y consume los mismos eventos vía broker.

Consecuencias - (+) La interacción transfers→notifications y transfers→audit queda desacoplada: caídas de MS1 o del monolito-audit no afectan la ejecución de transferencias. - (+) Demuestra los dos estilos de integración con justificación: síncrono donde hay dinero, asíncrono donde hay consistencia eventual aceptable. - (-) Aparecen estados intermedios visibles (PENDING, FAILED) en el historial de transferencias. - (-) Mayor complejidad operativa: outbox, reconciliador y broker pasan a ser parte del camino crítico de la trazabilidad.

ADR-003 — API Gateway: Spring Cloud Gateway

Estado: Aprobado

Contexto El Strangler Fig requiere un punto de entrada único que enrute /notifications/** y /transfers/** a los microservicios y el resto al monolito, de forma transparente (mismo host:puerto que el monolito original, 8080). Además, el Paso 4 exige observabilidad **también en el gateway** (spans propios y propagación W3C TraceContext).

Opciones evaluadas

Opción	Ventajas	Desventajas
Spring Cloud Gateway	Mismo stack del equipo (Spring Boot 3 / JVM): el agente OpenTelemetry lo instrumenta igual que al resto, dando spans y propagación de traza sin configuración extra. Rutas declarativas en YAML versionadas en git.	Consume más memoria que un proxy nativo.
Nginx	Liviano, maduro.	Instrumentación OTel requiere módulo externo; configuración fuera del ecosistema del equipo.
YARP	Excelente en .NET.	El equipo eligió el stack Java; introduce un runtime distinto sin beneficio.
Kong / Traefik	Features enterprise (rate limiting, plugins).	Sobredimensionado para 3 upstreams; curva de aprendizaje y otro componente operativo.

Decisión Spring Cloud Gateway, con tres rutas declarativas (notifications-ms, transfers-ms, monolith) configurables por variables de entorno. El switch de una ruta (rollback de una extracción) es un cambio de URI sin redeploy de los servicios.

Consecuencias - (+) Trazabilidad E2E desde el primer hop con el mismo agente OTel; un solo lenguaje operativo. - (+) El rollback de cada extracción es re-apuntar una ruta (decisión reversible, ver trade-offs). - (-) Punto único de fallo: en producción requeriría réplicas detrás de un LB. - (-) JVM en el edge: arranque más lento y mayor huella que Nginx/Envoy.

ADR-004 — Message Broker: Apache Kafka

Estado: Aprobado

Contexto La comunicación MS2 → MS1 y MS2 → monolito(audit) es asíncrona vía eventos. El dominio bancario impone: **durabilidad** (ningún evento de negocio puede perderse — trazabilidad regulatoria), **orden** por transferencia, y **replay** (re-auditarse histórico ante un requerimiento del regulador).

Opciones evaluadas

Opción	Ventajas	Desventajas
Kafka	Log durable y replicado; retención configurable y replay nativo por offset (clave regulatoria). Orden garantizado por partición (key = transferId). Consumer groups independientes: notifications y audit consumen el mismo evento sin interferirse. Métricas de consumer lag estándar (requisito Paso 4).	Operación más pesada (aunque KRaft eliminó ZooKeeper); curva de aprendizaje.
RabbitMQ	Routing flexible, ligero, DLX nativo.	El mensaje se borra al consumirse: sin replay nativo ni log histórico — débil para el requisito regulatorio.
SQS/SNS, Azure Service Bus	Cero operación.	Acopla la solución a un cloud; el reto corre local con Docker.
NATS JetStream	Muy ligero, con persistencia.	Ecosistema y tooling de auditoría menos maduros; menos conocido por el equipo.

Decisión Apache Kafka 3.7 (KRaft, un solo broker para la demo) con el tópico `finbank.transfers.events.v1` (key = transferId → orden por agregado) y su DLT `finbank.transfers.events.v1.dlt`. Consumer groups: `ms-notifications` y `monolith-audit`.

Consecuencias - (+) Replay y retención satisfacen la restricción regulatoria; el mismo evento alimenta N consumidores. - (+) Consumer lag por grupo expuesto vía `kafka-exporter` a Prometheus. - (-) Un broker = sin alta disponibilidad (aceptado para la demo; en producción: 3 brokers, RF=3, min.insync.replicas=2). - (-) Complejidad operativa nueva para el equipo (mitigada con `kafka-ui` en el compose).

ADR-005 — Base de datos de ms-notifications: PostgreSQL 16 dedicado

Estado: Aprobado

Contexto Database-per-Service: MS1 necesita una base exclusiva. El modelo del dominio es simple: una tabla `append-mostly` de notificaciones con un payload semiestructurado (JSONB) consultada por usuario.

Opciones evaluadas

Opción	Ventajas	Desventajas
PostgreSQL 16 (instancia dedicada)	El payload ya es JSONB en el monolito → backfill 1:1 sin transformación (menor riesgo de migración, ADR-011). JSONB cubre la parte documental. Conocimiento existente del equipo, mismas herramientas.	“Aburrido”: no aprovecha un modelo documental puro.

Opción	Ventajas	Desventajas
MongoDB	Natural para documentos de notificación.	Obliga a transformar datos en la migración y a operar un motor nuevo; ninguna consulta del dominio lo exige.
Redis	Ultrarrápido para feeds efímeros.	Las notificaciones bancarias deben persistir (evidencia ante reclamos); persistencia durable no es su fuerte.

Decisión PostgreSQL 16 en instancia dedicada (postgres-notifications, base notifications_db, schema equivalente al original + tabla processed_events para idempotencia). El criterio del reto es que la base sea **exclusiva y adecuada al modelo del dominio** — no que sea distinta por ser distinta.

Consecuencias - (+) Migración de datos = copia directa del schema notifications.* (sin ETL). - (+) Idempotencia transaccional del consumidor (notificación + processed_event en la misma tx ACID). - (-) Una instancia más que operar (costo aceptado del Database-per-Service).

ADR-006 — Base de datos de ms-transfers: PostgreSQL 16 dedicado

Estado: Aprobado

Contexto MS2 persiste el estado de la saga (PENDING/COMPLETED/FAILED/COMPENSATED) y el **outbox transaccional**. El Outbox Pattern exige que el cambio de estado y la inserción del evento ocurran en **una misma transacción ACID local** — este requisito define la elección.

Opciones evaluadas

Opción	Ventajas	Desventajas
PostgreSQL 16 (instancia dedicada)	Transacciones ACID multi-tabla → outbox correcto por construcción. Montos NUMERIC(19,4) exactos. Índice parcial (WHERE published_at IS NULL) eficiente para el polling del publisher. Compatible con CDC/Debezium si el outbox evoluciona a log tailing.	Nada exótico que mostrar.
MongoDB	Flexible.	Transacción estado+outbox requiere transacciones multi-documento (replica set) y Decimal128: más fricción para la misma garantía.
Event store (EventStore DB)	Audit trail por diseño.	Cambio de paradigma (event sourcing) no justificado: el reto ya cubre auditoría con el módulo audit + Kafka.

Decisión PostgreSQL 16 dedicado (postgres-transfers, base transfers_db: tablas transfers + outbox). El reto permite que MS2 use una base diferente a MS1 “si el modelo de datos lo justifica” — aquí ambos dominios son relacionales-transaccionales; elegir motores distintos sería diversidad sin justificación, lo documentamos explícitamente como decisión consciente.

Consecuencias - (+) Outbox Pattern con garantía ACID real (demo: broker caído → eventos retenidos → publicados al volver). - (+) Estados de la saga consultables con SQL para reconciliación y soporte. - (-) El polling del outbox introduce latencia de publicación (~1 s) — aceptable para notificación/auditoría.

ADR-007 — Patrón de consistencia distribuida: Saga orquestada con movimiento de dinero atómico

Estado: Aprobado

Contexto En el monolito, `TransferUseCase.execute()` era **una transacción ACID**: validar → debitar → acreditar → persistir transfer → notificar → auditar. Al extraer transfers como microservicio, esa atomicidad se rompe. La restricción regulatoria del reto: la consistencia del dinero **no es negociable**; hay que decidir conscientemente dónde aplica consistencia fuerte y dónde eventual.

Opciones evaluadas

Opción	Ventajas	Desventajas
Saga orquestada + movimiento débito/crédito atómico en accounts	El dinero nunca queda en estado intermedio: débito+crédito siguen siendo UNA tx ACID dentro del monolito (POST /internal/accounts/movements, idempotente). La saga solo coordina estado del transfer + efectos eventuales (notificación, auditoría). Orquestador único (ms-transfers) → lógica de compensación centralizada y estado consultable (clave regulatoria).	Acoplamiento síncrono ms-transfers → accounts: si accounts cae, no hay transferencias (CP sobre AP, asumido).
Saga coreografiada (debit event → credit event → ...)	Desacoplamiento máximo.	Dinero “en vuelo” entre débito y crédito; lógica de compensación dispersa; razonar el estado global es difícil — inaceptable con la restricción regulatoria y ambas cuentas viviendo en la MISMA base.
Two-Phase Commit (XA)	Atomicidad distribuida real.	Bloqueos distribuidos, coordinador como SPOF, soporte pobre en stacks modernos; descartado por fragilidad operativa.
Consistencia eventual también para saldos	Disponibilidad máxima.	Violación directa de la restricción del dominio: posibles saldos negativos o dobles débitos visibles al cliente.

Decisión Saga orquestada por ms-transfers con separación explícita de garantías:

Operación	Garantía	Mecanismo
Débito + crédito	Fuerte (ACID)	Una sola llamada idempotente a accounts (monolito), que ejecuta ambos en una tx
Estado del transfer	Saga: PENDING → COMPLETED / FAILED / COMPENSATED	Transiciones locales + outbox
Notificación / auditoría	Eventual, at-least-once	Eventos Kafka vía outbox; consumidores idempotentes

Operación	Garantía	Mecanismo
Caso borde (movimiento aplicado, estado no registrado)	Forward recovery	Reconciliador: consulta el movimiento por su idempotency key y completa o falla la saga
Reverso de una transferencia aplicada	Compensación	Movimiento inverso idempotente (comp-<transferId>) + evento TransferCompensated

Consecuencias - (+) Imposible el doble débito (idempotency key = transferId) y el dinero jamás queda a medias. - (+) Estado de cada transferencia consultable y auditable — el regulador puede reconstruir cualquier operación. - (–) Se sacrifica disponibilidad: accounts caído ⇒ 503 controlado (circuit breaker + fallback), no se aceptan transferencias “a ciegas”. - (–) El cliente puede ver una notificación llegar segundos después de la transferencia (consistencia eventual asumida y documentada).

ADR-008 — Arquitectura interna de los microservicios: Hexagonal pragmática (sin CQRS)

Estado: Aprobado

Contexto El reto exige elegir y justificar la arquitectura interna de cada microservicio (hexagonal, capas, CQRS, u otra). El monolito ya usaba capas domain / application / infrastructure / api con interfaces formales.

Opciones evaluadas

Opción	Ventajas	Desventajas
Hexagonal pragmática (ports & adapters sobre las capas existentes)	Continuidad cognitiva con el monolito. Puertos explícitos donde hay frontera real: AccountsPort aísla el caso de uso del HTTP/resiliencia; el outbox aísla la publicación de eventos del broker. Testeable con dobles de los puertos.	Menos “pura” que una hexagonal estricta (las entidades JPA hacen de dominio).
Capas clásicas sin puertos	Mínimo esfuerzo.	El caso de uso quedaría acoplado a RestClient/Kafka; cambiar el broker o el transporte tocaría la lógica de negocio.
CQRS	Modelos de lectura/escritura separados, escalado asimétrico.	Ninguno de los dos dominios tiene asimetría real: notifications es una lista por usuario; transfers escribe y lee lo mismo. Costo (sincronización de modelos, complejidad) sin beneficio medible.
Event Sourcing	Audit trail perfecto.	La auditoría ya está resuelta por el módulo audit + retención de Kafka; reconstruir estado por replay es un salto de paradigma injustificado.

Decisión Hexagonal pragmática en ambos microservicios: domain (entidades y estados), application (casos de uso + puertos: AccountsPort, repositorios), infrastructure (adaptadores: JPA, AccountsHttpAdapter con Resilience4j, OutboxPublisher/consumidores Kafka, seguridad JWT), api (controllers REST con el contrato original). **CQRS y Event Sourcing se descartan explícitamente** y quedan registrados como alternativas evaluadas.

Consecuencias - (+) La lógica de la saga (TransferUseCase) no conoce HTTP ni Kafka: se prueba contra puertos. - (+) Equipo productivo desde el día 1 (misma convención del monolito). - (-) Si mañana aparece un dashboard de consultas masivas, habrá que introducir un read model (decisión reversible: agregar CQRS después es factible).

ADR-009 — Stack de observabilidad: OpenTelemetry + Jaeger + Prometheus/Grafana + Loki

Estado: Aprobado

Contexto Paso 4: tres pilares (logs estructurados, métricas, trazas) en gateway, ambos microservicios y monolito remanente, con **un único Traceld propagado E2E incluyendo los mensajes del broker**, latencia P99, tasa de errores, throughput y consumer lag.

Opciones evaluadas

Opción	Ventajas	Desventajas
OTel Java agent (zero-code) + Collector + Jaeger + Prometheus + Grafana + Loki	Agnóstico y estándar de industria (recomendado por el propio reto). El agente instrumenta Spring MVC/WebFlux, RestClient, JDBC y Kafka (propaga traceparent en headers del mensaje) sin tocar código. Un solo pipeline (Collector) para los 3 pilares. Todo open source y local.	Tuning fino requiere conocer el agente.
Micrometer + Zipkin + ELK	Integración Spring nativa.	Tres integraciones distintas a mantener; Micrometer Tracing exige dependencias y config por servicio; ELK pesado para correr local.
Datadog / CloudWatch / X-Ray	Llave en mano.	Costo, cuenta cloud y lock-in; el reto corre en docker-compose local.

Decisión - Instrumentación: OpenTelemetry Java agent 2.5.0 adjunto vía -javaagent en los 4 servicios (variable JAVA_OPTS del compose) — cero código. - **Propagación:** W3C TraceContext en HTTP y en headers Kafka. En el outbox, el traceparent de la request original se persiste con el evento y el publisher lo restaura como header → la traza sobrevive al desacople temporal del outbox. - **Trazas:** Jaeger (OTLP). **Métricas:** Collector → Prometheus (+ kafka-exporter para consumer lag) → Grafana (dashboard provisionado: P99, errores, throughput, lag). **Logs:** LogstashEncoder (JSON a stdout con trace_id/span_id inyectados por el agente en el MDC) + export OTLP → Loki → Grafana.

Consecuencias - (+) Un Traceld correlaciona gateway → ms-transfers → monolito(accounts) → Kafka → ms-notifications / monolito(audit), y aparece en los logs JSON de todos. - (+) Cambiar de backend (p.ej. Jaeger→Tempo) es editar el Collector, no los servicios. - (-) ~50-80 MB extra de RAM por servicio por el agente y mayor tiempo de arranque (aceptado). - (-) El stack de observabilidad agrega 6 contenedores al compose (costo de la evidencia del Paso 4).

ADR-010 — Contrato de eventos: JSON con envelope CloudEvents y versionamiento explícito

Estado: Aprobado

Contexto Paso 3: los contratos de los eventos (schema, formato, versionamiento) deben quedar formalizados. Hay 3 tipos de evento (TransferCompleted, TransferFailed, TransferCompensated) y 3 consumidores en 2 procesos distintos.

Opciones evaluadas

Opción	Ventajas	Desventajas
JSON + envelope CloudEvents + versión en type y en el tópico	Legible y debuggeable (kafka-ui muestra el evento tal cual). Envelope estándar (specversion, id, type, source, time) + extensión traceparent. Sin infraestructura adicional.	Validación de schema no forzada por el broker (se valida por código y documentación).
Avro + Schema Registry	Evolución de schema forzada, payload compacto.	Un componente más (registry) y serialización binaria opaca para 3 eventos: complejidad > beneficio a esta escala. Deuda técnica asumida: si el catálogo crece, migrar a registry.
Protobuf	Compacto, tipado.	Mismo costo de tooling sin registry que lo gobierne; menos legible en demos.

Decisión Eventos JSON con envelope tipo CloudEvents 1.0:

```
{
  "specversion": "1.0",
  "id": "<uuid – idempotency key del consumidor>",
  "type": "finbank.transfers.TransferCompleted.v1",
  "source": "ms-transfers",
  "time": "2026-06-10T12:00:00Z",
  "traceparent": "00-<traceId>-<spanId>-01",
  "data": {
    "transferId": "uuid", "userId": "uuid",
    "sourceAccountId": "uuid", "targetAccountId": "uuid",
    "amount": "150.00", "currency": "USD",
    "reference": "string", "status": "COMPLETED",
    "reason": "string (solo Failed/Compensated)"
  }
}
```

Reglas de versionamiento: cambios compatibles (agregar campos opcionales a data) no cambian versión; cambios incompatibles crean ...TransferCompleted.v2 en el tópico finbank.transfers.events.v2, con doble publicación durante la ventana de migración de consumidores. Catálogo completo en docs/eventos.md.

Consecuencias - (+) id del envelope funciona como idempotency key uniforme en todos los consumidores. - (+) Eventos legibles en demos y debugging; contratos versionados en git. - (-) Sin enforcement automático de schema: un productor puede romper el contrato sin que el broker lo impida (mitigado por catálogo + consumidores tolerantes a campos desconocidos; deuda técnica documentada).

ADR-011 — Estrategia de migración de datos sin downtime: Expand → Migrate → Contract

Estado: Aprobado

Contexto Cada módulo extraído debe llevarse su schema (notifications.*, transfers.*) del PostgreSQL compartido a su base exclusiva **sin downtime** y sin perder trazabilidad (restricción regulatoria).

Opciones evaluadas

Opción	Ventajas	Desventajas
Expand/Migrate/Contract con doble escritura + backfill	Sin downtime; rollback instantáneo en cada fase (re-apuntar la ruta del gateway); verificable (conteos/checksums antes del switch).	Ventana de doble escritura que operar y monitorear.
Big-bang (parar, copiar, arrancar)	Simple.	Downtime explícito — prohibido por el requisito.
CDC con Debezium	Doble escritura sin tocar código (lee el WAL).	Un componente más (Kafka Connect); excelente opción de evolución, innecesaria para el volumen actual.

Decisión Expand → Migrate → Contract por módulo:

1. **Expand:** levantar la DB exclusiva del microservicio con su schema Flyway propio (estructura 1:1 con el schema original → copia directa sin ETL).
2. **Doble escritura:** el monolito escribe en su schema y replica al nuevo destino (o CDC si se prefiere no tocar código). Las lecturas siguen en el monolito.
3. **Backfill + verificación:** copia histórica (pg_dump --schema=notifications | psql ...) con la doble escritura activa; verificación por conteos y checksums por rango de fechas.
4. **Switch de lecturas:** el gateway re-apunta /notifications/** (luego /transfers/**) al microservicio. **Rollback = revertir la ruta** (segundos).
5. **Contract:** retirar la doble escritura; congelar el schema original en read-only durante el periodo de validación regulatoria; eliminarlo al vencer.

Para transfers, el corte es más simple: las transferencias son inmutables una vez terminadas — basta backfill histórico + redirigir las *nuevas* transferencias al MS (los registros nuevos nacen en la DB nueva); el histórico se migra en frío lógico mientras el servicio sigue arriba.

Alcance de la demo: las bases del compose nacen vacías (no hay datos productivos que migrar); la estrategia queda documentada aquí y el switch transparente del gateway queda demostrado.

Consecuencias - (+) Cero downtime y rollback barato en cada fase; evidencia verificable para el regulador. - (+) Estructuras 1:1 hacen el backfill trivial y auditable. - (-) Durante la doble escritura existen dos fuentes de verdad temporales (se mitiga declarando autoritativa la del monolito hasta el switch). - (-) Los schemas congelados ocupan espacio hasta la fase contract (aceptado: respaldo regulatorio).

ADR-012 — Microfrontends: composición con Module Federation (Webpack 5)

Estado: Aprobado

Contexto El punto extra pide un portal donde el frontend refleje la separación de dominios del backend: un App Shell que componga **en tiempo de ejecución** un MFE por cada microservicio extraído, con despliegue independiente, JWT compartido sin exponer credenciales y al menos un MFE mostrando estado en tiempo real.

Opciones evaluadas

Opción	Ventajas	Desventajas
Module Federation (Webpack 5)	Integración en runtime real (remoteEntry.js se descarga al navegar, el bundle del shell no contiene código de los MFEs); dependencias compartidas negociadas (react singleton); permite frameworks distintos por MFE; mismo enfoque visto en clase.	Acopla la toolchain a Webpack 5; los contratos compartidos (nombre del remote, módulo expuesto) deben versionarse con disciplina.
iframes	Aislamiento total (CSS/JS/seguridad).	UX pobre (routing, modales, resize), comunicación torpe vía postMessage, y el JWT habría que pasarlo a cada frame explícitamente.
Web Components	Estándar del browser, agnóstico de framework.	No resuelve la carga remota en runtime por sí solo (habría que construirla a mano) ni el sharing de dependencias.
Single-SPA	Orquestación madura multi-framework.	Capa adicional de framework por encima del bundler; Module Federation ya cubre la necesidad con menos piezas.

Decisión Module Federation con la misma topología del backend:

- **Shell (React 18, host):** login contra /auth (solo el shell maneja credenciales), routing global y composición; carga notifications/NotificationsApp y transfers/TransfersApp con React.lazy/import() dinámico.
- **mfe-notifications (React 18, remote):** dominio del MS1; consume GET /notifications.
- **mfe-transfers (Vue 3, remote):** dominio del MS2 en **framework distinto** (demuestra autonomía de equipos); bridge mount/unmount hacia el shell. Tiempo real vía **SSE** (GET /transfers/stream): cada transición de la saga (PENDING/COMPLETED/FAILED/COMPENSATED) llega al browser al instante.
- **Proxy del portal (nginx, :8085):** sirve shell y MFEs bajo el mismo origen y enruta /api/** al API Gateway → **sin CORS** y el token nunca cruza orígenes.

Compartición del JWT (criterio de seguridad) El token vive **solo en memoria** en window.__FINBANK_SESSION__ (store mínimo con subscribe/update), nunca en localStorage ni cookies legibles por JS persistentes: un refresh exige re-login, pero no quedan credenciales persistidas robables. Los MFEs leen el token del store; solo el shell conoce el flujo de login/registro.

Comunicación entre MFEs (sin imports cruzados) CustomEvent('finbank:transfer-update') en window: lo emite mfe-transfers al recibir un evento SSE; lo escuchan mfe-notifications (refresca su lista cuando el

evento llega por Kafka a MS1) y el navbar del shell (badge de último estado). Acoplamiento: solo el nombre del evento.

SSE y autenticación EventSource no permite headers → el JWT viaja como query param access_token solo en /transfers/stream. Implicación: el token puede quedar en logs de acceso del proxy; mitigaciones aplicadas/posibles: TTL corto del token, endpoint sin caché, y en producción se migraría a cookie HttpOnly por sesión SSE o a un ticket de un solo uso.

Consecuencias - (+) Despliegue independiente real: docker compose up -d --build portal-mfe-transfers reconstruye un MFE sin tocar shell ni el otro; si un MFE cae, un error boundary del shell degrada solo esa ruta. - (+) El portal replica el mapa de dominios del backend: el equipo de transfers es dueño de su servicio y de su UI. - (-) Cuatro contenedores más y una toolchain frontend (Webpack/babel) que mantener. - (-) shared con strictVersion exige coordinar upgrades de React entre shell y mfe-notifications (fallar rápido fue elección deliberada frente a duplicar el framework).

Análisis de Trade-offs (Paso 5)

1. ¿La consistencia eventual es aceptable para todas las operaciones bancarias de FinBank?

No, y la arquitectura lo refleja explícitamente. Clasificamos cada operación:

Operación	Consistencia	Justificación
Débito + crédito de saldos	Fuerte (ACID) — nunca eventual	Un saldo intermedio visible (débito sin crédito) o un doble débito es una violación regulatoria y de confianza. Por eso el movimiento permanece como una transacción única en accounts y la saga jamás lo parte (ADR-007).
Validación de fondos y propiedad	Fuerte	Decidir con datos viejos permitiría sobregiros; se valida dentro de la misma tx del movimiento.
Notificación al cliente	Eventual (at-least-once)	Que la alerta llegue 2 segundos tarde no tiene impacto financiero; idempotencia evita duplicados.
Auditoría regulatoria	Eventual sin pérdida	El regulador exige completitud, no inmediatez: outbox + Kafka durable garantizan que todo evento llega, aunque sea tarde.
Historial de transferencias	Eventual de lectura	El estado PENDING visible es información honesta del proceso, no una inconsistencia.

2. ¿En qué operaciones se sacrificó disponibilidad para mantener consistencia?

En la **ejecución de transferencias**. Si accounts (monolito) no responde, ms-transfers NO acepta la transferencia “para procesarla luego”: el circuit breaker abre y responde 503 controlado, y el transfer queda FAILED sin movimiento de dinero. Elegimos CP sobre AP para dinero: una transferencia aceptada sin verificar fondos podría rebotar después de que el cliente la cree completada. En contraste, para notificaciones/auditoría elegimos AP: ms-notifications caído **no** detiene transferencias (el evento espera en Kafka/outbox).

3. ¿Qué tan difícil sería revertir las decisiones? ¿Cuáles son reversibles?

Decisión	Reversibilidad	Cómo
Ruta del gateway hacia un MS (ADR-003)	Trivial	Re-apuntar la URI al monolito (variable de entorno); minutos.
Extracción de notificaciones (ADR-001)	Fácil mientras el schema congelado exista	El código del módulo vive en git; restaurar ruta + replicar datos de vuelta.
Extracción de transfers (ADR-002)	Media	Hay que restaurar las llamadas in-process y reconciliar el histórico de la DB nueva.
Kafka como broker (ADR-004)	Media-difícil	Productores/consumidores están detrás de outbox/listeners (cambiar el adaptador), pero replay y offsets son semántica Kafka.
Saga + idempotencia + outbox (ADR-007)	Prácticamente irreversible (y no se quería)	Es la semántica del negocio distribuido; quitarla reintroduce los modos de fallo que elimina.
PostgreSQL en los MS (ADR-005/006)	Media	Migración de datos estándar; el código depende de JPA, no del motor.

La estrategia general (Strangler Fig + gateway) fue elegida precisamente porque **mantiene barata la reversibilidad en cada paso** — a diferencia de un big-bang.

4. ¿El incremento en complejidad operativa está justificado para el tamaño actual de FinBank?

Honestamente: **parcialmente**. Pasamos de 2 contenedores a ~14. La justificación no es la carga actual sino: (a) el driver del negocio es regulatorio y de evolución independiente de los dominios, no throughput; (b) los costos son mayormente fijos y automatizados (compose levanta todo; la observabilidad reduce el costo de diagnóstico que antes era grep en un log); (c) el costo se pagó incrementalmente y cada fase dejó el sistema funcionando. Lo que NO estaría justificado hoy: extraer los 5 módulos, Schema Registry, Kubernetes — por eso no se hizo (ver pregunta 5 y deuda técnica en ADR-010).

5. ¿Hubo módulos que se decidió NO extraer? ¿Por qué?

Sí, tres, deliberadamente:

- **accounts:** es el núcleo de consistencia fuerte. Extraerlo hoy convertiría cada validación de saldo en una llamada remota y distribuiría la única transacción que exige ACID. Se extraería al final, cuando exista una necesidad real de escalarlo, idealmente con su propio equipo. Mientras tanto, su interfaz interna (/internal/accounts/movements) ya define el contrato que tendría como servicio — el camino queda preparado.
- **audit:** el reto pide demostrar que módulos remanentes consumen eventos sin ser extraídos, y audit es el caso ideal: pasó de llamada in-process a consumidor Kafka idempotente sin salir del monolito. Beneficio de extraerlo hoy: ninguno; riesgo regulatorio: alto.
- **auth:** emite y valida credenciales; es transversal. Migrarlo requeriría una estrategia de identidad propia (¿IdP dedicado? ¿Keycloak?) que merece su propia iniciativa, no un apéndice de este reto.

El criterio rector: **se extrae cuando hay un driver concreto (dependencia del paso anterior, tolerancia a consistencia eventual, beneficio de despliegue independiente), no por completar un inventario.**

